# HERRAMIENTAS DE ETL EN ENTORNOS BIG DATA

## BIG DATA APLICADA

**Elaborado por:**

**CARDENAS ACARO, CLAUDIA MILAGROS**

**DAMIANI KAEMENA, STEPHANI**

**Solicitado por:**

**SERGIO ORIZANO SALVADOR**

**Link GitHub:**

**https://github.com/steph-d989/IDL1_BDA_DAMIANI_STEPHANI.git**

**Perú, 2026**

# Contenido


**1. INTRODUCCIÓN 3**

**2. OBJETIVO GENERAL 3**

**3. OBJETIVOS ESPECIFICOS 3**

**4. DESARROLLO DEL PROYECTO 4**

4.1. CREACION DE REPOSITORIO 4

4.2. CREACION DE ENTORNO VIRTUAL 5

4.3. PREPARACION DE ENTORNO DE TRABAJO 5

4.4. CREACION DE PROYECTO EN SUPABASE 6

4.5. DISEÑO EN LA ARQUITECTURA DE DATOS 8

4.6. CREACION DE ARCHIVO CON CREDENCIALES Y DEPENDENCIAS 11

4.7. CONEXION DE BASE DE DATOS Y ENTORNO DE DESARROLLO 11

4.8. VERIFICAR CONEXIÓN DE BASE DE DATOS Y REVISION DE ARCHIVOS 13

**5. RESULTADOS OBTENIDOS 14**

**6. CONCLUSIONES 14**

# HERRAMIENTAS DE ETL EN ENTORNOS BIG DATA

## 1. INTRODUCCIÓN

Este proyecto desarrolla el proceso de implementación de un sistema de Big Data orientado al almacenamiento, organización y procesamiento de datos, este proyecto integra buenas prácticas de control de versiones, gestión de entornos de trabajo, seguridad de credenciales y conexión a una base de datos en la nube mediante supabase.

Se utilizó una estructura basada en el sistema Medallion, con esquemas Bronze, Silver y Gold, esto permite estructurar la información de acuerdo con su nivel de procesamiento.

El desarrollo de este proyecto nos permitirá entender el uso de herramientas ETL y los entornos de Big Data que contribuyen a la organización eficiente del flujo de los datos, desde su extracción hasta su transformación.

## 2. OBJETIVO GENERAL

Implementar un sistema de Big Data para el almacenamiento y procesamiento de datos, con las herramientas de desarrollo, control de versiones y base de datos en la nube, nos permite manejar una estructura organizada, segura y escalable.

## 3. OBJETIVOS ESPECIFICOS

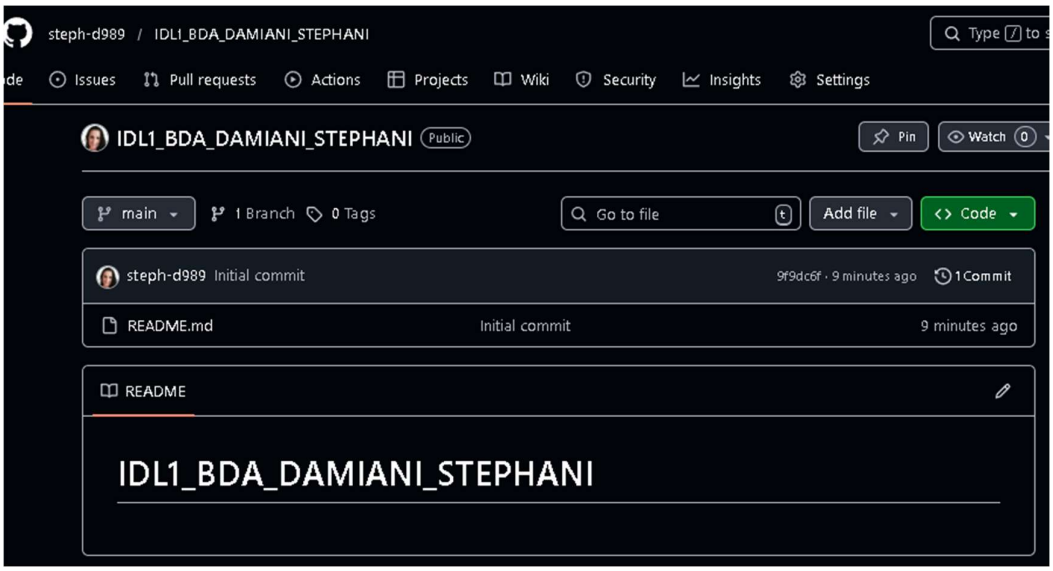
- Crear y configurar un repositorio.
- Preparar un entorno virtual para desarrollo en Python.
- Implementar buenas prácticas en el manejo de versiones.
- Configurar una base de datos en Supabase.

- Diseñar una arquitectura de datos en esquemas BRONZE, SILVER y GOLD.
- Establecer una conexión entre VS Code y Supabase.
- Verificar el acceso a los archivos.

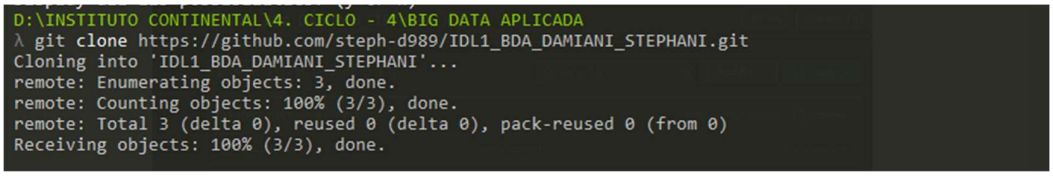
# 4. DESARROLLO DEL PROYECTO

## 4.1. CREACION DE REPOSITORIO

En una primera etapa, se realizó la creación del repositorio de GitHub, incorporando el archivo readme, para poder documentar el proyecto desde el principio.




Se clonó el repositorio, en el equipo local, lo cual nos permite trabajar de manera ordenada y mantenernos sincronizados con el repositorio remoto, lo cual nos asegura la trazabilidad de cambios y facilidad del trabajo colaborativo.



Una vez clonado el repositorio ingresamos a la carpeta debe figurar el main -> origin.

```
D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA
λ cd IDL1_BDA_DAMIANI_STEPHANI
```

## 4.2. CREACION DE ENTORNO VIRTUAL

Se implementó un entorno virtual, para así poder aislar las dependencias del proyecto y evitar conflictos con otros proyectos, una vez creado se activa, y se valida que el entorno este habilitado antes de abrir el VS Code. Esta práctica nos permite una administración más segura y controlada.

```
D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA\IDL1_BDA_DAMIANI_STEPHANI (main -> origin)
λ python -m venv venv_bda_idl1
```

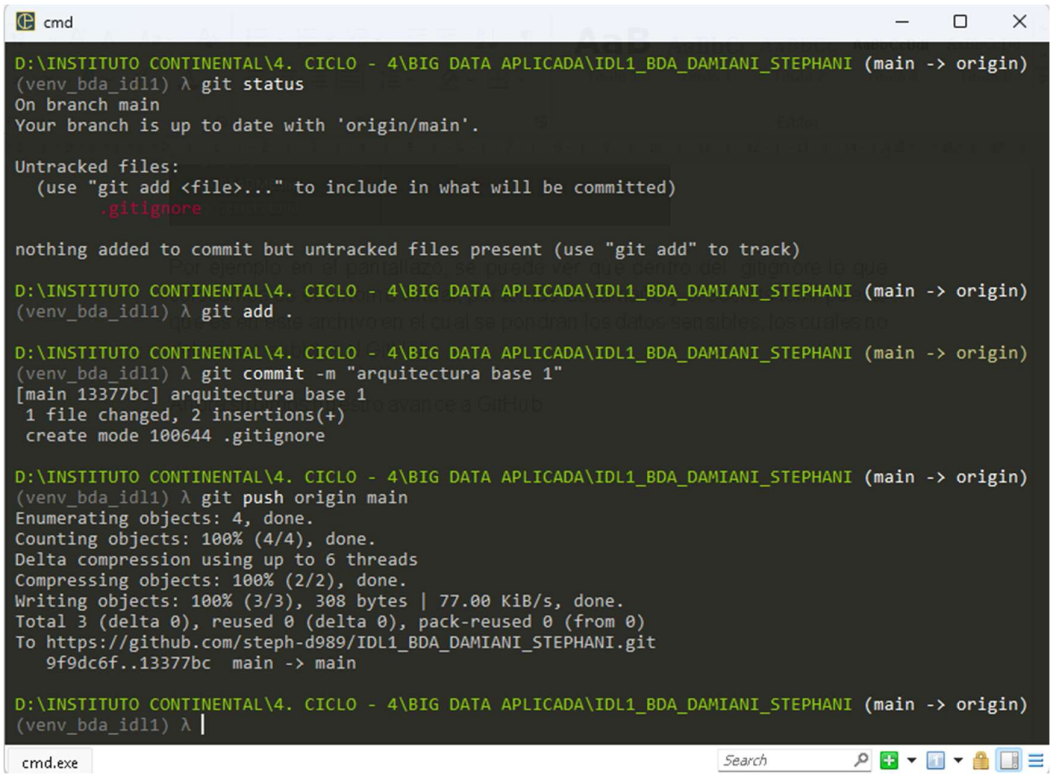
```
D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA\IDL1_BDA_DAMIANI_STEPHANI (main -> origin)
λ venv_bda_idl1\Scripts\activate.bat

D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA\IDL1_BDA_DAMIANI_STEPHANI (main -> origin)
(venv_bda_idl1) λ
```

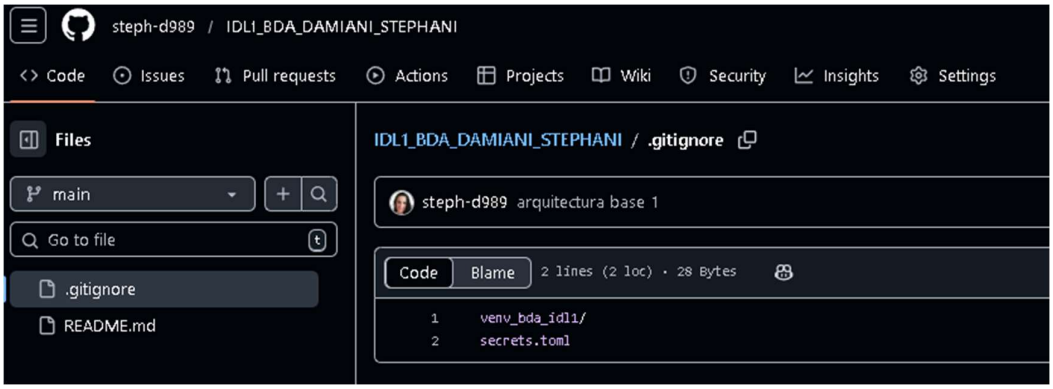
## 4.3. PREPARACION DE ENTORNO DE TRABAJO

Configuramos él .gitignore para excluir del control de versiones los elementos que no deben de compartirse en el repositorio, habitualmente por su tamaño o por la sensibilidad de informa con que se guarda en ellos. Entre los archivos que no se suben al repositorio se encuentran los secrets.toml, que poseen información sensible o el entorno virtual que es un archivo muy pesado.

Asimismo, se realizaron los primeros envíos de avance al repositorio remoto, con los comandos:

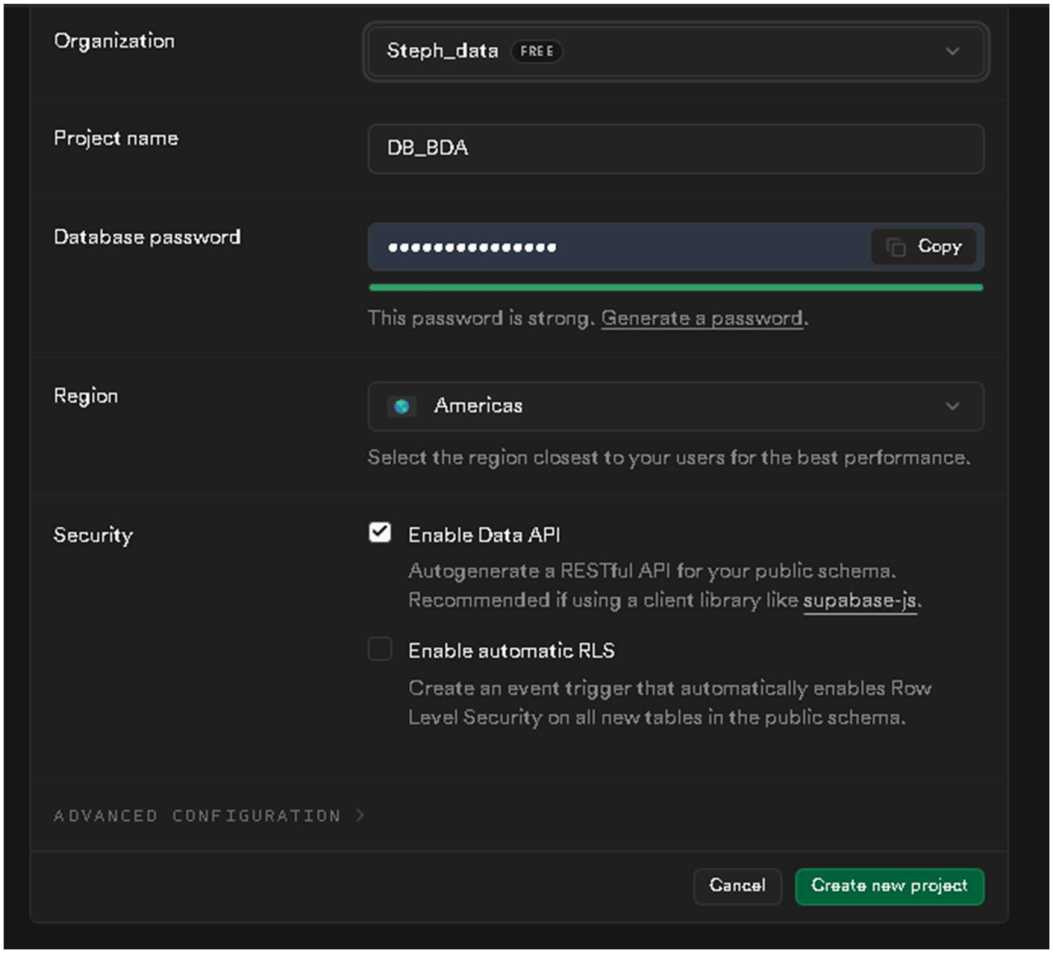
- `Git status`: Ver que archivos aún no se subieron a Git
- `Git add .`: Indicar que todos los archivos modificados se subirán a Git.
- `Git commit -m “nombre de commit”`: Indicar el nombre del commit.
- `Git push origin main`: Enviar a Git todas las modificaciones.

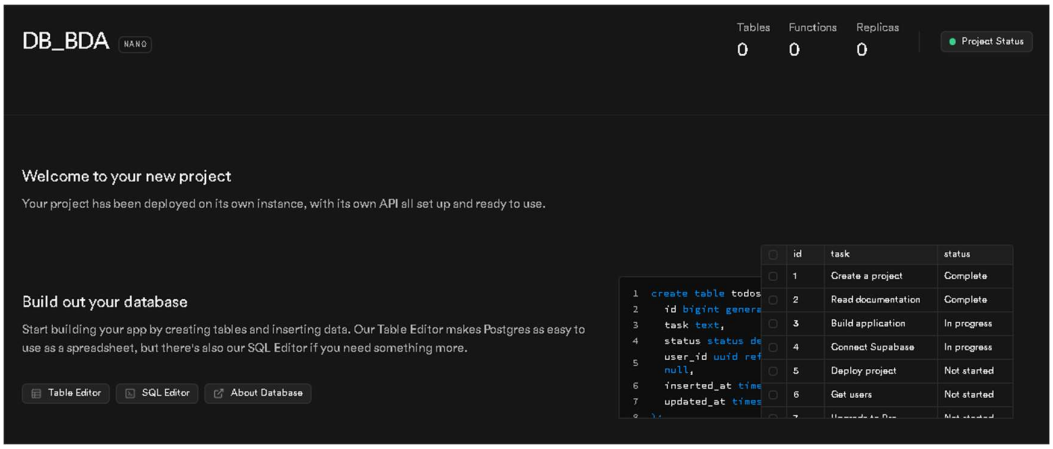


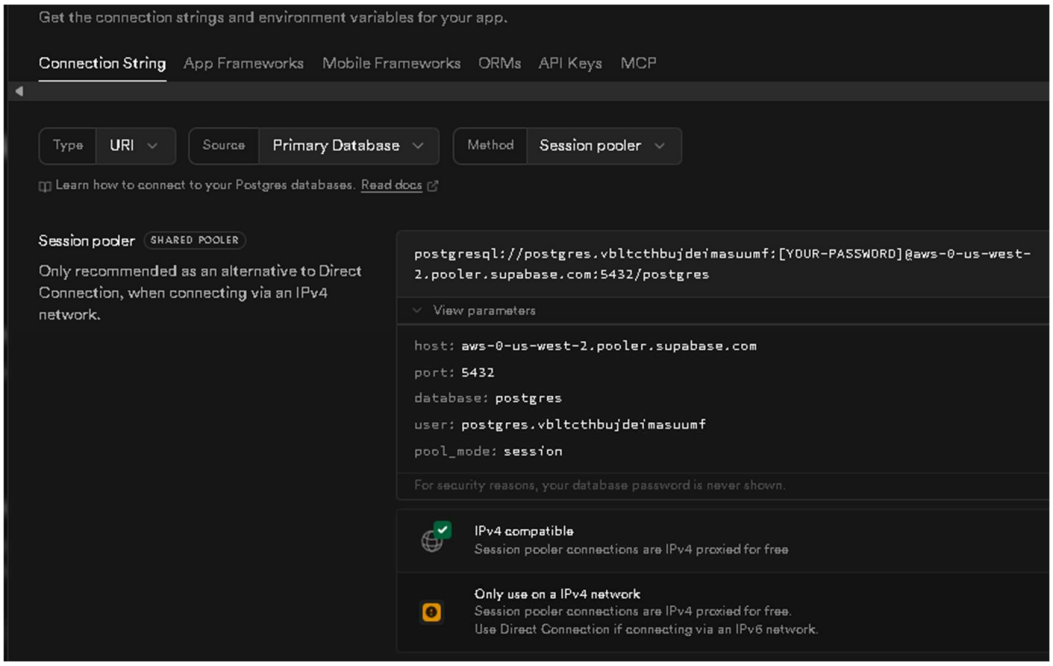




## 4.4. CREACION DE PROYECTO EN SUPABASE

En la siguiente fase, creamos un proyecto en Supabase, registrando parámetros de conexión necesarios para el enlace con la base de datos. Accedimos a la sección de connect, utilizando el método Session Pooler, desde donde obtuvimos los datos necesarios para la configuración del entrono local.



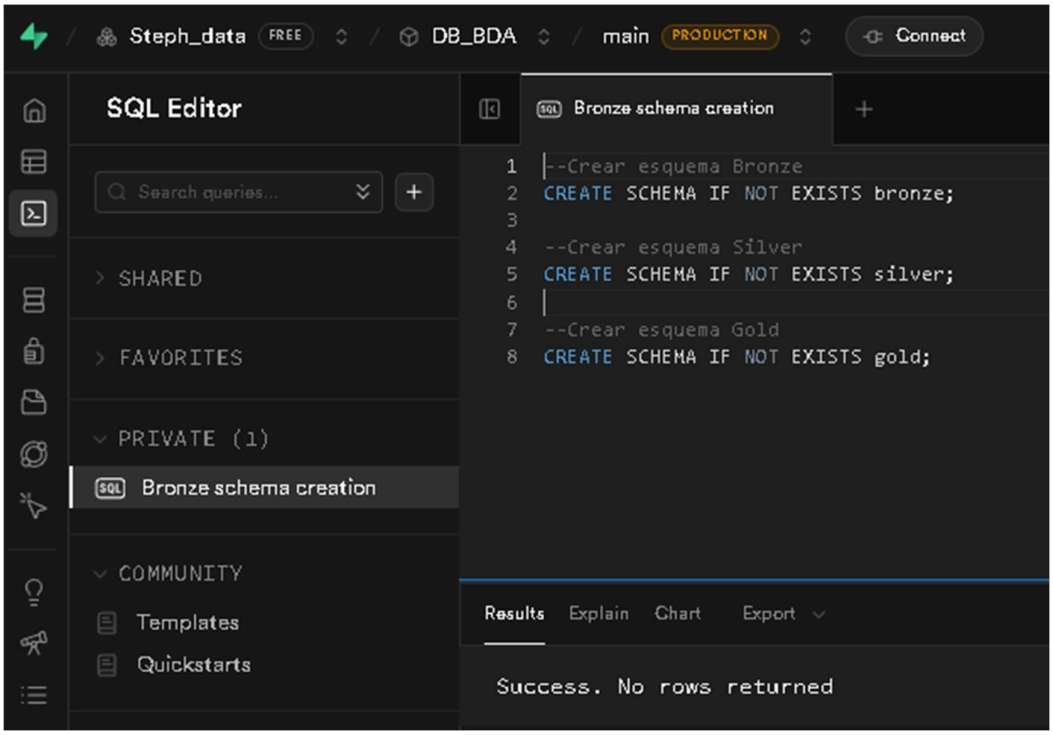


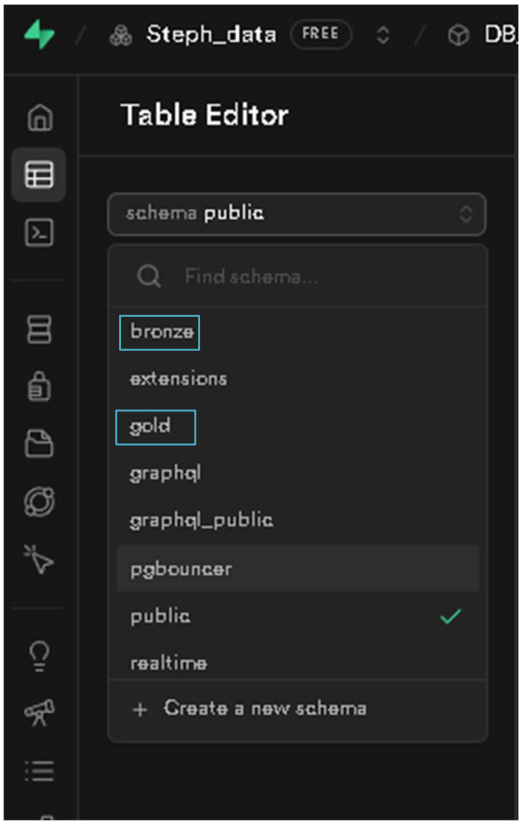

## 4.5. DISEÑO EN LA ARQUITECTURA DE DATOS

Dentro de Supa base, se implementaron los esquemas Bronze, Silver y Gold desde el SQL Editor. Esta estructura, responde a una arquitectura en la que:

- Bronze: Almacena los datos en bruto, tal como se extraen de la fuente.
- Silver: Contiene datos depurados y transformados.
- Gold: Reúne información procesada y optimizada para el análisis o consumo final.

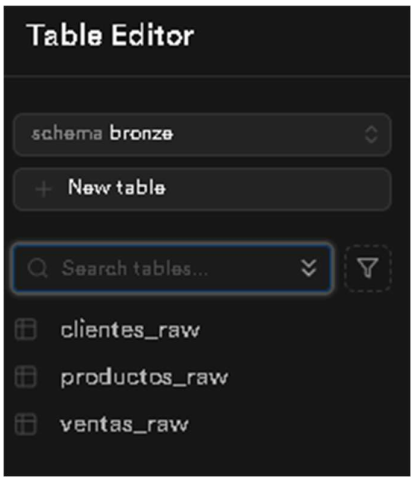
Después se verificó la creación de estos esquemas y de las tablas asociadas a través del TABLE Editor. Esta organización llamada Medallion, es fundamental en Big Data, puesto que permite mantener un orden, estabilidad y más calidad al momento de tratar los datos.


Steph_data
FREE
DB_BDA
main
PRODUCTION
Connect
SQL Editor
Bronze schema creation
Search queries...
SHARED
FAVORITES
PRIVATE (1)
Bronze schema creation
COMMUNITY
Templates
Quickstarts
--Crear esquema Bronze
CREATE SCHEMA IF NOT EXISTS bronze;
--Crear esquema Silver
CREATE SCHEMA IF NOT EXISTS silver;
--Crear esquema Gold
CREATE SCHEMA IF NOT EXISTS gold;
Results
Explain
Chart
Export
Success. No rows returned


Steph_data
FREE
Table Editor
schema public
Find schema...
bronze
extensions
gold
graphql
graphql_public
pgbouncer
public
realtime
Create a new schema

```sql
--Crear tabla en el esquema bronze
CREATE TABLE bronze.ventas_raw(
  id_venta TEXT,
  fecha TEXT,
  id_cliente TEXT,
  id_producto TEXT,
  cantidad INT,
  precio_unitario FLOAT,
  total FLOAT,
  canal TEXT,
  region TEXT,
  fuente_archivo TEXT, --para saber de que archivo version
  fecha_carga TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE TABLE bronze.productos_raw(
  id_producto TEXT,
  nombre_producto TEXT,
  categoria TEXT,
  subcategoria TEXT,
  precio FLOAT,
  proveedor TEXT,
  fecha_carga TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE bronze.clientes_raw(
  id_cliente TEXT,
  nombre_cliente TEXT,
  genero TEXT,
  edad TEXT,
  ciudad TEXT,
  pais TEXT,
  segmento TEXT,
  fecha_carga TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



## 4.6. CREACION DE ARCHIVO CON CREDENCIALES Y DEPENDENCIAS

Instalamos las librerías requeridas para el proyecto dentro del entorno virtual, para habilitar la conexión de datos. Luego generamos el archivo requirements.txt, esto para registrar las dependencias utilizadas. Lo que garantiza que cualquier persona que trabaje con el repositorio pueda replicar el proyecto en su propio entorno de trabajo, esto mejora la portabilidad y el trabajo en equipo.

```
cmd
D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA\IDL1_BDA_DAMIANI_STEPHANI (main -> origin)
(venv_bda_idl1) λ pip install psycopg2-binary python-dotenv
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.11-cp312-cp312-win_amd64.whl.metadata (5.1 kB)
Collecting python-dotenv
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Downloading psycopg2_binary-2.9.11-cp312-cp312-win_amd64.whl (2.7 MB)
   ---------------------------------------- 2.7/2.7 MB 11.5 MB/s eta 0:00:00
Downloading python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Installing collected packages: python-dotenv, psycopg2-binary
Successfully installed psycopg2-binary-2.9.11 python-dotenv-1.2.2

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
D:\INSTITUTO CONTINENTAL\4. CICLO - 4\BIG DATA APLICADA\IDL1_BDA_DAMIANI_STEPHANI (main -> origin)
(venv_bda_idl1) λ pip freeze > requirements.txt
```

```
IDL1_BDA_D_
  .secrets
    __pycache__
    db_config.py
    secrets.toml
  assets\docs
    clientes.csv
    productos.csv
    ventas_2024.csv
    ventas.csv
  notebooks
    carga_bronze.ip...  U
  venv_bda_idl1
  .gitignore  M
  README.md
  requirements.txt  U
```

```
requirements.txt
numpy==2.4.2
pandas==3.0.1
psycopg2-binary==2.9.11
python-dateutil==2.9.0.post0
python-dotenv==1.2.2
six==1.17.0
tomlkit==0.10.2
tzdata==2025.3
```

## 4.7. CONEXION DE BASE DE DATOS Y ENTORNO DE DESARROLLO

Se procedió a configurar la conexión entre la base de datos de Supabase y el proyecto desarrollado en VS Code. Para ello, se definieron las rutas necesarias y se preparó el archivo de

configuraciones, que contiene los parámetros de acceso. Esta etapa es esencial para permitir la interacción entre el código y la base de datos.

```python
import psycopg2     #Conexion de postgres con python
import os       #Navega por carpetas de la compu
from dotenv import load_dotenv #Lee cualquier archivo env

# CARGAR VARIABLES DEL ARCHIVO .ENV
load_dotenv()

def get_connection():
    return psycopg2.connect(
        host = "aws-0-us-west-2.pooler.supabase.com",
        port = 5432,
        database = "postgres",
        user = "postgres.vbltcthbujdeimasuumf",
        password = "WaoQueSegura123"
        )
```

carga_bronze.ipynb U | secrets.toml | db_config.py

notebooks > carga_bronze.ipynb > with open("../.secrets/secrets.toml",

Generate + Code + Markdown | Colab | Run All Resta

```python
import pandas as pd
import os
import sys
import tomllib

sys.path.append(os.path.abspath("../.secrets"))
from db_config import get_connection

print("Se conectó a la bbdd correctamente...")
```

[6] ✓ 0.0s

```
Se conectó a la bbdd correctamente...
```

carga_bronze.ipynb U | secrets.toml | db_config.py

.secrets > secrets.toml

```toml
[paths]
bronze_path = "assets/docs"
```

```
with open("../.secrets/secrets.toml", "rb") as f:
    config = tomllib.load(f)

ruta_archivos = config["paths"]["bronze_path"]

print("Ruta configurada: ", ruta_archivos)
✓ 00s
```

```
Ruta configurada:  assets/docs
```

## 4.8. VERIFICAR CONEXIÓN DE BASE DE DATOS Y REVISION DE ARCHIVOS

Finalmente, realizamos la verificación de la conexión con la base de datos, para esto, se definieron las rutas necesarias y se preparó el archivo de configuración, el cual contiene los parámetros de acceso. Esta etapa permite la interacción entre el código y la base de datos, lo que facilita las operaciones de consulta, carga, edición y validación de información.

```
conn = get_connection()
cursor = conn.cursor()

print("Conexión Exitosa")
✓ 14s
```

```
Conexión Exitosa
```

```python
archivos = [f for f in os.listdir(f"../{ruta_archivos}") if f.endswith(".csv")]
print(f"\n Archivos disponibles: ")
for i, archivo in enumerate(archivos):
    print(f"{i} - {archivo}")

indice = int(input(f"\n Selecciona el numero del archivo que deseas cargar: "))

if indice < 0 or indice >= len(archivos):
    print("Número invalido")
    exit()

archivo_seleccionado = archivos[indice]
print(f"\n Cargando archivo: {archivo_seleccionado}")
```
✓ 3.7s

```
 Archivos disponibles:
0 - clientes.csv
1 - productos.csv
2 - ventas.csv
3 - ventas_2024.csv

 Cargando archivo: ventas_2024.csv
```

## 5. RESULTADOS OBTENIDOS

Como resultado del Desarrollo, logramos establecer una base funcional para un sistema de Big Data, con una estructura organizada desde el repositorio hasta la base de datos. Se implementó un entorno de trabajo con la posibilidad de ser replicado, de manera segura y escalable, con un almacenamiento por niveles que facilita y mejora la gestión de los datos, y se consolidó la conexión entre el entorno local y la plataforma la plataforma de SupaBase. Lo cual permitió sentar las bases para futuros procesos ETL.

## 6. CONCLUSIONES

La implementación de este Sistema, nos permite la integración de herramientas fundamentales para la administración moderna de datos, combinando control de versiones, entornos virtuales, seguridad de credenciales y el almacenamiento en la nube de las bases de datos.

La adaptación del proyecto a esquemas Bronze, Silver y Gold nos aporta una estructura idónea para organizar la información en base a su nivel de procesamiento.

El proyecto también evidencia la importancia de seguir las buenas prácticas, especialmente en lo que respecta a la replicación del proyecto en entornos diferentes y la protección de información sensible. Este proyecto nos da una base solidad para continuar con la automatización de procesos a partir de ETL.